МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«КАЗАНСКИЙ (ПРИВОЛЖСКИЙ) ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт вычислительной математики и информационных технологий
Кафедра прикладной математики и искусственного интеллекта

ОТЧЕТ
Многослойный персептрон
Введение в машинное обучение

Выполнила:
студентка гр. 09-322
Бородина А. С.

Проверила:
преподаватель
Гиниятова Д. Х.

Казань 2026

1. Цель и задачи

Целью данной работы является практическое освоение архитектуры искусственных нейронных сетей прямого распространения (Multilayer Perceptron, MLP) и алгоритмов их обучения без использования высокоуровневых фреймворков.

Основные задачи:

1. Проведение первичного анализа и предобработки набора данных data.csv.
2. Реализация мок-модуля разделения данных на обучающую и валидационную выборки со стандартизацией признаков.
3. Математический вывод и самостоятельная программирование алгоритмов:
    - Прямого распространения;
    - Вычисления функции потерь;
    - Обратного распространения ошибки;
    - Оптимизации методом градиентного спуска.
4. Оценка эффективности модели и визуализация кривых обучения.

2. Математические основы и алгоритмы.

2.1. Прямое распространение

Для каждого слоя l вычисляется линейная комбинация входов и весов:

$$z^{[l]} = A^{[l-1]}W^{[l]} + b^{[l]};$$

где:

$A^{[l-1]} -$ матрица активаций предыдущего слоя (для $l = 1, A^{[0]} = X$);

$W^{[l]}$ матрица весов слоя l;

$b^{[l]} -$ вектор смещений ($bias$) слоя l.

Далее применяется функция активации. В скрытых слоях используется Sigmoid:

$$\sigma(Z) = \frac{1}{1 + e^{-Z}};$$

На выходном слое применяется функция Softmax для получения вероятностного распределения по классам ($C = 2$):

$$Softmax(Z_k) = \frac{e^{Z_k}}{\sum_{j=1}^{C} e^{Z_j}};$$

2.2. Функция потерь

В качестве функции потерь используется Бинарная / Категориальная Кросс-Энтропия:

$$\mathcal{L}(Y, \hat{Y}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{C} Y_{i,k} \log(\hat{Y}_{i,k});$$

где m – размер батча, Y – One-Hot кодированные истинные метки, $\hat{Y}$ – предсказанные вероятности Softmax.

2.3. Обратное распространение ошибки

Градиенты рассчитываются последовательно от выходного слоя к входному с использованием цепного правила дифференцирования.

1. Градиент на выходном слое L (при сочетании Softmax + Cross-Entropy):

$$\frac{\partial \mathcal{L}}{\partial Z^{[L]}} = \delta^{[L]} = \hat{Y} - Y;$$

2. Градиенты весов и смещений слоя l:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \frac{1}{m}\left(A^{[l-1]}\right)^T \delta^{[l]};$$

$$\frac{\partial \mathcal{L}}{\partial b^{[l]}} = \frac{1}{m}\sum_{i=1}^{m} \delta_{i,:}^{[l]};$$

3. Перенос ошибки на предыдущий скрытый слой $l - 1$:

$$\delta^{[l-1]} = \left(\delta^{[l]}\left(W^{[l]}\right)^T\right) \odot \sigma'\left(Z^{[l-1]}\right);$$

где $\sigma'(Z) = \sigma(Z) \odot \big(1 - \sigma(Z)\big)$ – производная сигмоиды, а $\odot$ – поэлементное произведение (Hadamard product).

2.4. Обновление параметров

Обновление параметров происходит по правилу градиентного спуска с темпом обучения α (learning_rate):

$$W^{[l]} \leftarrow W^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial W^{[l]}};$$

$$b^{[l]} \leftarrow b^{[l]} - \alpha \frac{\partial \mathcal{L}}{\partial b^{[l]}};$$

3. Реализация

Файл	Назначение
split.py	загрузка CSV, стратифицированное разбиение, расчёт и применение нормировки
mlp.py	функции активации, softmax, кросс-энтропия, инициализация; классы DenseLayer и MLP
train.py	цикл обучения, лог по эпохам, сохранение модели, графики
predict.py	загрузка модели, прогноз, BCE и метрики
eda.py	текстовый анализ данных

Результаты сохраняются в папки: data/ (train.csv, valid.csv, scaler.pkl), models/model.pkl, results/ (train_log.txt, metrics.txt), results/plots/ (loss.png, accuracy.png). Папки создаются автоматически. Полный код приведён в приложении.

Слой `DenseLayer` хранит веса W и смещения b, запоминает вход x и выход a при прямом проходе; метод backward_z вычисляет градиенты:

```
def forward(self, x):
    self.x = x
    z = x @ self.W + self.b
```

```
        self.a = softmax(z) if self.activation == 'softmax' else
ACTIVATIONS[self.activation][0](z)
        return self.a

    def backward_z(self, dz):
        self.gW = self.x.T @ dz
        self.gb = dz.sum(axis=0, keepdims=True)
        return dz @ self.W.T
```

Сеть `MLP` собирает слои в цепочку; обратный проход начинается с (P − Y)/N:

```
    def backward(self, Y):
        grad = (self.layers[-1].a - Y) / Y.shape[0]
        grad = self.layers[-1].backward_z(grad)
        for layer in reversed(self.layers[:-1]):
            grad = layer.backward(grad)

    def train_step(self, X, Y, lr, momentum=0.0):
        self.forward(X)
        self.backward(Y)
        for layer in self.layers:
            layer.update(lr, momentum)
```

Цикл обучения (train.py) на каждой эпохе перемешивает данные, обучает сеть по мини-батчам, затем считает loss и accuracy на обучающей и проверочной выборках и печатает строку вида epoch 100/100 | train_loss=… val_loss=… | train_acc=… val_acc=….

Запуск:

```
python split.py
python train.py
python predict.py
```

4. Эксперименты и результаты

Параметры итоговой модели: архитектура 30-16-16-2, ReLU, He-инициализация, 100 эпох, η = 0,0001, размер батча 1, seed = 42.

4.1. Динамика обучения

Эпоха	train_loss	val_loss	train_acc	val_acc
1	0,9789	0,8784	0,3969	0,3982
5	0,6554	0,6138	0,5724	0,6372
10	0,5080	0,4956	0,8465	0,8319
25	0,3090	0,3104	0,9276	0,9115
50	0,1718	0,1825	0,9518	0,9381
75	0,1238	0,1404	0,9737	0,9558
100	0,1016	0,1221	0,9803	0,9558

В ходе обучения сделаны графики, отражающие динамику Loss и Accuracy:

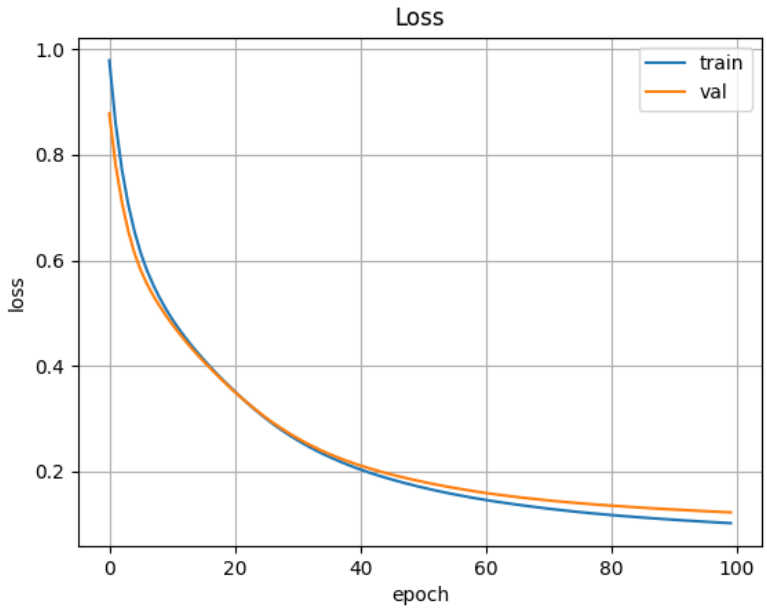


Рисунок 1. Функция потерь на обучающей (train) и проверочной (val) выборках.

Демонстрирует плавное монотонное убывание ошибки без признаков переобучения (Overfitting).

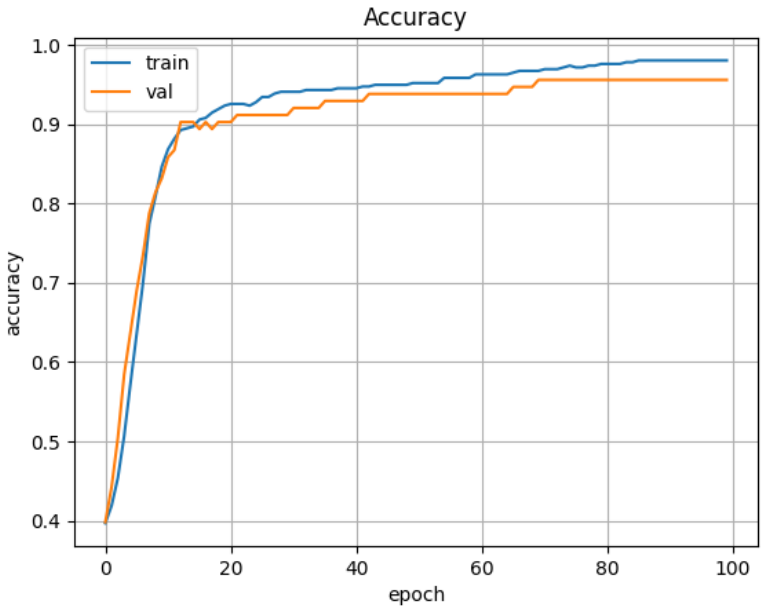


Рисунок 2. Точность классификации на обучающей (train) и проверочной (val) выборках.

Показывает быстрый рост точности до плато в районе 40-50 эпохи.

4.2. Итоговое качество на проверочной выборке

Метрика	Значение
Бинарная кросс-энтропия	0,1221
Accuracy	0,9558
Precision (класс M)	0,9512
Recall (класс M)	0,9286
F1	0,9398

Матрица ошибок (M – положительный класс):

	Прогноз B	**Прогноз M**
Истина B	69 (TN)	2 (FP)
Истина M	3 (FN)	39 TP)

5. Заключение

В ходе выполнения лабораторной работы была успешно реализована гибкая модульная система многослойного персептрона на языке Python без использования сторонних фреймворков глубокого обучения.

- Правильная предварительная обработка данных и инициализация весов играют решающую роль в предотвращении проблемы затухания градиентов при использовании функции Sigmoid.
- Реализованный алгоритм обратного распространения ошибки демонстрирует высокий уровень точности на данной задаче.

6. Листинг

split.py

```
import argparse
import os
import pickle
import numpy as np


def load_csv(data):
    labels, feats = [], []
    with open(data) as f:
        for line in f:
            p = line.strip().split(',')
            if len(p) < 32:
                continue
            labels.append(1 if p[1] == 'M' else 0)
            feats.append([float(v) for v in p[2:]])
    return np.array(feats), np.array(labels)


def split_dataset(data, seed, valid_size, out_train, out_valid):
    with open(data) as f:
        rows = [l for l in f.read().splitlines() if l.strip()]
    labels = np.array([r.split(',')[1] for r in rows])
    rng = np.random.RandomState(seed)
    tr, va = [], []
    for c in np.unique(labels):
        idx = rng.permutation(np.where(labels == c)[0])
        n_val = int(round(len(idx) * valid_size))
        va += idx[:n_val].tolist()
        tr += idx[n_val:].tolist()
    tr, va = rng.permutation(tr), rng.permutation(va)
    for name, ix in ((out_train, tr), (out_valid, va)):
        os.makedirs(os.path.dirname(name) or '.', exist_ok=True)
        with open(name, 'w') as f:
```

```
            f.write('\n'.join(rows[i] for i in ix) + '\n')
    print(f'train: {len(tr)} -> {out_train} | valid: {len(va)} -> {out_valid}
(seed={seed})')


def fit_scaler(X):
    return X.mean(axis=0), X.std(axis=0) + 1e-12


def standardize(X, mean, std):
    return (X - mean) / std


def load_scaler(data):
    with open(data, 'rb') as f:
        return pickle.load(f)


def main():
    ap = argparse.ArgumentParser(description='Разделение data.csv и расчёт
параметров нормировки')
    ap.add_argument('--dataset', default='data.csv')
    ap.add_argument('--seed', type=int, default=42)
    ap.add_argument('--valid_size', type=float, default=0.2)
    ap.add_argument('--out_train', default=os.path.join('data', 'train.csv'))
    ap.add_argument('--out_valid', default=os.path.join('data', 'valid.csv'))
    ap.add_argument('--scaler', default=os.path.join('data', 'scaler.pkl'))
    r = ap.parse_args()
    split_dataset(r.dataset, r.seed, r.valid_size, r.out_train, r.out_valid)
    X, _ = load_csv(r.out_train)
    mean, std = fit_scaler(X)
    with open(r.scaler, 'wb') as f:
        pickle.dump({'mean': mean, 'std': std}, f)
    print(f'> параметры нормировки (по train) сохранены: {r.scaler}')


if __name__ == '__main__':
    main()
```

mlp.py

```
import os
import pickle
import numpy as np
```

```
EPS = 1e-15


def sigmoid(z): return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))
def relu(z): return np.maximum(0, z)
def tanh(z): return np.tanh(z)

ACTIVATIONS = {
    'sigmoid': (sigmoid, lambda a: a * (1 - a)),
    'relu': (relu, lambda a: (a > 0).astype(float)),
    'tanh': (tanh, lambda a: 1 - a ** 2),
}


def softmax(z):
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)


def binary_cross_entropy(Y, P):
    P = np.clip(P, EPS, 1 - EPS)
    return float(-np.mean(Y * np.log(P) + (1 - Y) * np.log(1 - P)))


def init_weights(n_in, n_out, how, rng):
    if how == 'heUniform':
        lim = np.sqrt(6.0 / n_in)
        return rng.uniform(-lim, lim, (n_in, n_out))
    if how == 'heNormal':
        return rng.randn(n_in, n_out) * np.sqrt(2.0 / n_in)
    if how == 'xavier':
        lim = np.sqrt(6.0 / (n_in + n_out))
        return rng.uniform(-lim, lim, (n_in, n_out))
    raise ValueError(f'неизвестная инициализация: {how}')


class DenseLayer:

    def __init__(self, n_in, n_out, activation='relu', initializer='heUniform',
rng=None):
        rng = rng if rng is not None else np.random.RandomState(0)
        self.activation = activation
        self.W = init_weights(n_in, n_out, initializer, rng)
        self.b = np.zeros((1, n_out))
```

```
        self.vW, self.vb = np.zeros_like(self.W), np.zeros_like(self.b)

    def forward(self, x):
        self.x = x
        z = x @ self.W + self.b
        self.a = softmax(z) if self.activation == 'softmax' else
ACTIVATIONS[self.activation][0](z)
        return self.a

    def backward_z(self, dz):
        self.gW = self.x.T @ dz
        self.gb = dz.sum(axis=0, keepdims=True)
        return dz @ self.W.T

    def backward(self, da):
        return self.backward_z(da * ACTIVATIONS[self.activation][1](self.a))

    def update(self, lr, momentum=0.0):
        self.vW = momentum * self.vW - lr * self.gW
        self.vb = momentum * self.vb - lr * self.gb
        self.W += self.vW
        self.b += self.vb


class MLP:
    def __init__(self, sizes, activation='relu', initializer='heUniform', seed=0):
        """sizes = [n_in, h1, h2, ..., n_out]. Скрытые слои - activation, выход -
softmax."""
        self.sizes, self.activation, self.initializer = list(sizes), activation, initializer
        rng = np.random.RandomState(seed)
        self.layers = []
        for i, (a, b) in enumerate(zip(sizes[:-1], sizes[1:])):
            act = 'softmax' if i == len(sizes) - 2 else activation
            self.layers.append(DenseLayer(a, b, act, initializer, rng))

    def forward(self, X):
        for layer in self.layers:
            X = layer.forward(X)
        return X

    predict_proba = forward

    def backward(self, Y):
        grad = (self.layers[-1].a - Y) / Y.shape[0]
        grad = self.layers[-1].backward_z(grad)
```

```
        for layer in reversed(self.layers[:-1]):
            grad = layer.backward(grad)

    def train_step(self, X, Y, lr, momentum=0.0):
        self.forward(X)
        self.backward(Y)
        for layer in self.layers:
            layer.update(lr, momentum)

    def get_weights(self):
        return [(l.W.copy(), l.b.copy()) for l in self.layers]

    def set_weights(self, weights):
        for l, (W, b) in zip(self.layers, weights):
            l.W, l.b = W.copy(), b.copy()

    def save(self, path, mean, std):
        os.makedirs(os.path.dirname(path) or '.', exist_ok=True)
        with open(path, 'wb') as f:
            pickle.dump({'sizes': self.sizes, 'activation': self.activation, 'initializer':
self.initializer,
                         'weights': self.get_weights(), 'mean': mean, 'std': std}, f)

    @staticmethod
    def load(path):
        with open(path, 'rb') as f:
            d = pickle.load(f)
        net = MLP(d['sizes'], d['activation'], d['initializer'])
        net.set_weights(d['weights'])
        return net, d['mean'], d['std']
```

train.py

```
import argparse
import os
import numpy as np
from mlp import MLP, ACTIVATIONS, binary_cross_entropy
from split import load_csv, standardize, load_scaler


def accuracy(P, y):
    return float((P.argmax(axis=1) == y).mean())
```

```
def fit(net, Xt, yt, Xv, yv, epochs, lr, batch_size, momentum=0.0, patience=0,
seed=0, log=print):
    rng = np.random.RandomState(seed)
    Yt, Yv = np.eye(2)[yt], np.eye(2)[yv]
    hist = {k: [] for k in ('loss', 'val_loss', 'acc', 'val_acc')}
    best, best_w, wait = np.inf, None, 0
    for ep in range(1, epochs + 1):
        order = rng.permutation(len(Xt))
        for s in range(0, len(Xt), batch_size):
            ix = order[s:s + batch_size]
            net.train_step(Xt[ix], Yt[ix], lr, momentum)
        Pt, Pv = net.predict_proba(Xt), net.predict_proba(Xv)
        hist['loss'].append(binary_cross_entropy(Yt, Pt))
        hist['val_loss'].append(binary_cross_entropy(Yv, Pv))
        hist['acc'].append(accuracy(Pt, yt))
        hist['val_acc'].append(accuracy(Pv, yv))
        log(f"epoch {ep:03d}/{epochs} | train_loss={hist['loss'][-1]:.4f}
val_loss={hist['val_loss'][-1]:.4f}"
            f" | train_acc={hist['acc'][-1]:.4f} val_acc={hist['val_acc'][-1]:.4f}")
        if patience:                                  # early stopping по val_loss
            if hist['val_loss'][-1] < best - 1e-6:
                best, wait, best_w = hist['val_loss'][-1], 0, net.get_weights()
            else:
                wait += 1
                if wait >= patience:
                    log(f'early stopping на эпохе {ep}, возвращаем веса лучшей эпохи')
                    break
    if best_w:
        net.set_weights(best_w)
    return hist


def plot_history(h, out_dir):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    os.makedirs(out_dir, exist_ok=True)
    for key, val_key, title, ylabel, name in (('loss', 'val_loss', 'Loss', 'loss', 'loss.png'),
                                              ('acc', 'val_acc', 'Accuracy', 'accuracy',
'accuracy.png')):
        plt.figure()
        plt.plot(h[key], label='train')
        plt.plot(h[val_key], label='val')
        plt.title(title); plt.xlabel('epoch'); plt.ylabel(ylabel)
        plt.legend(); plt.grid(True)
```

```
        plt.savefig(os.path.join(out_dir, name), dpi=100)
        plt.close()


def main():
    ap = argparse.ArgumentParser(description='Обучение многослойного
персептрона')
    ap.add_argument('--train', default=os.path.join('data', 'train.csv'))
    ap.add_argument('--valid', default=os.path.join('data', 'valid.csv'))
    ap.add_argument('--scaler', default=os.path.join('data', 'scaler.pkl'))
    ap.add_argument('--layer', type=int, nargs='+', default=[16, 16], help='размеры
скрытых слоёв')
    ap.add_argument('--activation', default='relu', choices=ACTIVATIONS)
    ap.add_argument('--initializer', default='heUniform', choices=['heUniform',
'heNormal', 'xavier'])
    ap.add_argument('--loss', default='binaryCrossentropy',
choices=['binaryCrossentropy'])
    ap.add_argument('--epochs', type=int, default=100)
    ap.add_argument('--batch_size', type=int, default=1)
    ap.add_argument('--learning_rate', type=float, default=0.0001)
    ap.add_argument('--momentum', type=float, default=0.0)
    ap.add_argument('--patience', type=int, default=0, help='early stopping (0 =
выкл.)')
    ap.add_argument('--seed', type=int, default=42)
    ap.add_argument('--model', default=os.path.join('models', 'model.pkl'))
    ap.add_argument('--plots', default=os.path.join('results', 'plots'))
    ap.add_argument('--log', default=os.path.join('results', 'train_log.txt'))
    r = ap.parse_args()

    os.makedirs(os.path.dirname(r.log) or '.', exist_ok=True)
    logfile = open(r.log, 'w', encoding='utf-8')

    def log(msg):
        print(msg)
        logfile.write(msg + '\n')

    sc = load_scaler(r.scaler)
    Xt, yt = load_csv(r.train)
    Xv, yv = load_csv(r.valid)
    Xt, Xv = standardize(Xt, sc['mean'], sc['std']), standardize(Xv, sc['mean'],
sc['std'])
    log(f'x_train shape : {Xt.shape}\nx_valid shape : {Xv.shape}')

    net = MLP([Xt.shape[1], *r.layer, 2], r.activation, r.initializer, r.seed)
```

```
    hist = fit(net, Xt, yt, Xv, yv, r.epochs, r.learning_rate, r.batch_size, r.momentum,
r.patience, r.seed, log)
    net.save(r.model, sc['mean'], sc['std'])
    log(f'saved to {r.model}')
    plot_history(hist, r.plots)
    log(f'saved to {r.plots}')
    logfile.close()


if __name__ == '__main__':
    main()
```

predict.py

```
import argparse
import os
import numpy as np
from mlp import MLP, binary_cross_entropy
from split import load_csv, standardize


def confusion_metrics(y, pred):
    tp = int(((pred == 1) & (y == 1)).sum()); tn = int(((pred == 0) & (y == 0)).sum())
    fp = int(((pred == 1) & (y == 0)).sum()); fn = int(((pred == 0) & (y == 1)).sum())
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    return dict(acc=(tp + tn) / len(y), precision=prec, recall=rec, f1=f1, tp=tp, tn=tn,
fp=fp, fn=fn)


def main():
    ap = argparse.ArgumentParser(description='Прогноз и оценка модели')
    ap.add_argument('--dataset', default=os.path.join('data', 'valid.csv'))
    ap.add_argument('--model', default=os.path.join('models', 'model.pkl'))
    ap.add_argument('--metrics', default=os.path.join('results', 'metrics.txt'))
    r = ap.parse_args()

    net, mean, std = MLP.load(r.model)
    X, y = load_csv(r.dataset)
    P = net.predict_proba(standardize(X, mean, std))
    m = confusion_metrics(y, P.argmax(axis=1))
    lines = [f'BCE loss: {binary_cross_entropy(np.eye(2)[y], P):.4f}',
             f'accuracy: {m['acc']:.4f} | precision: {m['precision']:.4f} | recall:
{m['recall']:.4f} | F1: {m['f1']:.4f}',
```

```
            f"confusion (M=positive): TP={m['tp']} FN={m['fn']} FP={m['fp']}
TN={m['tn']}"]
    print('\n'.join(lines))
    os.makedirs(os.path.dirname(r.metrics) or '.', exist_ok=True)
    with open(r.metrics, 'w', encoding='utf-8') as f:
        f.write('\n'.join(lines) + '\n')


if __name__ == '__main__':
    main()
```